

CS 744: Design and Engineering of Computing Systems Autumn 2026 Lab 1: tipping the first domino

Instructions

- This course is on a ***no-plagiarism*** diet. All parties involved in plagiarism harakiri will be referred to DADAC and penalized to the maximum extent.
- The Moss Detective Agency has agreed to conduct all investigations.
<https://theory.stanford.edu/~aiken/moss/>
Byomkesh, Sherlock, Phryne, Marple, and Hercule are on standby.
- Hardcoding expected output of execution in source code will yield **negative** marks.
- Generative AI (ChatGPT, Gemini, etc.) is not you!
Analytical, critical and creative thinking are learning outcomes of this course and source code or any content via GenAI tools cannot be part of your submissions.
- **Note:** Submission guidelines to be strictly followed; otherwise, your submission will not count.
- Login with username your **CSE LDAP ID** or **labuser** on software lab machines for this lab.
- This file is part of the **lab1.tar.gz** archive which contains multiple directories with programs associated with the exercise questions listed below.
- Most questions of this lab are tutorial style and are aimed to provide an introduction to tools and files related to system and process information and control.

Task 1: Setup

The following setup and tools are essential —

(i) A Unix/Linux machine

Options include setting up a Linux-based work environment on your personal machine or a virtual machine, WSL on Windows, or use a machine in one of the department computing labs (SL1, SL2, SL3).

```
$ssh username@sl2-23.cse.iitb.ac.in
```

username is your CSE LDAP ID.

SL2 machines are numbered and you can try numbers other than 23 as well.

(ii) Software tools

gcc – The GNU C compiler

gdb – The GNU debugger

make – GNU make utility to maintain groups of programs

A **text editor** (vi, emacs, sublime, vscode, ...)

The first task is to get access/setup a working environment and make sure that you have access to the basic software toolset. Also, get familiar with using **ssh** and its various flags so that you can login to the department machines and work. If you are working with a custom installation on a local machine, make sure the above tools are installed appropriately.

Task 2: Hello world!

Write a program whose executable binary depends on three source files.

hw (the executable/binary) depends on

hw.c
helloworld.h
helloworld.c

hw.c – is the driver program with the main function

helloworld.h – the function declarations header file

helloworld.c – the function definitions source file

Note: All these files are present in the folder **task2/helloworld/**

2.a. Compile the main program correctly so that it can execute and print the “Hello world!” message.

2.b. Create a **Makefile** in **task2/helloworld/** to set up the compilation process and dependencies among the source files. Further, modify the function to print “Hello world! CS744 begins.”

Many examples and tutorials are available online on the topics of Makefile. Here is one such examples — <https://www.oreilly.com/library/view/managing-projects-with/0596006101/ch01.html>

Make sure the makefile has the capabilities to correctly detect all dependencies and recompile the executable to correctly mimic changes to the source code. Test using modifications/updates to different source files. Also, add the capability within the makefile to clean up the binary and intermediate files using a label.

2.c. Another program **hu.c** in **task2/hellouniverse/** depends on three source files **hu.c**, **helloworld.c**, **helloworld.h**.

To make sure **hu** (the executable/binary) is compiled correctly, update the **Makefile** in **task2/helloworld/** to generate a **helloworld.o** object file.

Then create another **Makefile** in **task2/hellouniverse/** to compile the **hu.c**. Additionally, create a **Makefile** in **task2/** directory that lets you compile/clean the entire task (all executables).

Task 3: Some basic Linux tools

The following are some basic Linux tools, and the goal is to familiarize oneself with their usage and capabilities.

To know more about them use: `man <command>`. Start with `man man`.

- **top**

`top` provides a continuous collective view of the system and operating system state. For example — list of all processes, resource consumption of each process, system-level CPU usage etc. The system summary information displayed, order etc. has several configurable knobs.

`top` also allows you to send signals to processes (change priority, stop etc.).

Sample tasks: (Hint: `man top`)

- Display processes of specific user (e.g., labuser)
- Add **ppid** information to the displayed information (What is **pid**? What is **ppid**?)

- **ps**

The `ps` command is used to view the processes running on a system. It provides a snapshot of the processes along with detailed per process information like process-id, cpu usage, memory usage, command name etc. Several has several flags to display different types of process information, e.g., executing `ps` without arguments will not show all processes on the system, but a combination of flags as input parameters will.

Sample tasks: (Hint: `man ps`)

- List all the processes (of all users) in the system
- List all process whose **ppid** is 2
- Which process has **pid** 2 ?
- See options to view any process with a user-defined format

- **iostat**

`iostat` is a command useful for monitoring and reporting CPU and device usage statistics. For example, the command reports total activity and rate of activities (read/write) to each disk/partition and can be configured to monitor continuously (after every specified interval).

Sample tasks: (`man iostat`)

- Display average cpu utilization of your system
- Display average disk and cpu utilization every 3 seconds for 10 times

- **strace**

strace is a diagnostic and debugging tool used to monitor the interactions between processes and the operating system (Linux). The tool traces the set of functions (system calls/calls of the Application Binary Interface) and signals (events) used by a program to communicate with the operating system.

Sample task: (**man strace**)

- Display all system calls and signals made by any command (for example display the system calls made by **ls** command).
- Display summary of total time taken by each system call, time taken per system call during a program execution and the number of times a system call was invoked.

- **lsof**

lsof is a tool used to list open files. The tool lists details of the file itself and details of users, processes which are using the files.

Sample task: (**man lsof**)

- Display all opened files of a specific user.

- **lsblk**

lsblk is a tool used to list information about all available block devices such as hard disks drives (HDD), solid-state drive (SDD), flash drives, CD-ROM etc.

Sample task:

- Display all device permissions (read,write,execute) and owners.

- Also look up the following commands/tools:

pstree, lshw, lspci, lscpu, dig, netstat, df, du, watch

- **The proc file system**

The **proc** file system is a mechanism provided by Linux, for communication between userspace and the kernel (operating system) using the file system interface. Files in the **/proc** directory report values of several OS parameters and also can be used for configuration and (re)initialization. The **proc** file system is very well documented in the man pages, — **man proc**

Understand the system-wide proc files such as meminfo, cpuinfo, etc. and process related files such as status, stat, limits, maps etc. System related proc files are available in the directory **/proc**, and process related proc files are available at **/proc/<process-id>/**

Exercises

1. Collect the following basic information about your machine using the `proc` file system and the tools listed above and answer the following questions. Also, mention the tool and file you used to get the answers.
 - a. Find the Architecture, Byte Order and Address Sizes of your CPU.
 - b. How many CPU sockets, cores, and CPU threads does the machine have?
 - c. Find the sizes of L1, L2 and L3 cache.
 - d. What is the total main memory and secondary memory of your machine and how much of it is free?
 - e. Find the number of total, running, sleeping, stopped and zombie processes.
A zombie process is a stopped/terminated process waiting to be cleaned up.
 - f. How many context switches has the system performed since bootup? A context switch is the process of storing the state of a process or thread so that it can be restored and resume execution at a later point, and then restoring a different, previously saved, state. This allows multiple processes to share a single CPU and is an essential feature of a multitasking operating system.
2. Run all programs in the subdirectory named `task3/memory` and identify the memory usage of each program. Compare the memory usage of these programs in terms of `VmSize` & `VmRSS` and justify your observations based on the code.
3. Run the executable `subprocesses` provided in the sub-directory `task3/subprocess` and provide your roll number as a command line argument. Find the number of subprocesses created by this program. Describe how you obtained the answer.
4. Run `strace` along with the binary program of `empty.c` (file located in subdirectory `task3/strace`). What do you think the output of `strace` indicates in this case? How many different system calls can you identify?

Next, use `strace` along with the binary program of `hello.c` (which is in the same directory). Compare the two `strace` outputs,

- Which part of the output is common, and which part has to do with the specific program?
 - List all unique system calls for each program and look up the functionality of each.
5. Run the executable `openfiles` in subdirectory `task3/files`. List the files which are opened by this program, and describe how you obtained the answer.
 6. Find all the block devices on your system, their mount points and file systems present on them. A mount point is a file system directory entry from where a disk can be accessed. A file system describes how data is organized on a disk. Describe how you obtained the answer.

Task 4: Debugging with GDB

GNU Debugger, which is also called **gdb**, is the most popular debugger for UNIX systems to debug C/C++ programs. GDB can only be used to find runtime or logical errors. Please note that compile time errors are detected by the compiler and not a debugger.

Many documents and tutorials are available online that explain the capabilities, syntax and usage of gdb. Here is one document you can refer to:

<https://sourceware.org/gdb/current/onlinedocs/gdb.pdf>

In this part of the task you will use **gdb** to debug a program with simple logical errors and a runtime error.

How to use GDB?

Compile your program using the **-g** flag. It is particularly important as it generates debugging information that gdb can use.

```
gcc <filename> -g -o <executable_name>
```

GDB is invoked with the shell command **gdb**. Once started, it reads commands from the terminal until you tell it to exit with the GDB command **quit**.

Run GDB using the following command

```
gdb <executable_name>
```

The command above will attach gdb to your executable and open the gdb shell in which you can write commands to debug your program.

Some of the most frequently used GDB commands are:

break, run, bt, print, c, next, step, list, help, quit

To know more about them, use: **man gdb**

Task 4 does not involve writing any code, but you must submit a pdf/textfile with a brief description of what commands you used to debug the code followed by any additional comments.

Task 4a: Compile and run the **hw.c** (used in task2) program using GDB, and make use of various GDB commands to explore and understand its functionality.

Task 4b: Use **fibonacci.c** (present in `task4/`) to compile and execute a program.

This program is supposed to print fibonacci numbers until a certain value of **n**. However, there is a logical error in the code which causes it to print the wrong output. Your task is to use GDB to debug the program. You must insert suitable breakpoints, pause program execution, print intermediate values of variables from GDB, and monitor the execution step by step in order to find the logical error. Even if you can identify the error without stepping through the code, you must be able to demonstrate the process of debugging using GDB during your evaluation.

Task 4c: Debug the `pointers.c` program (present in `task4/`) using GDB. The program contains some pointer related operations. A bug is most likely generating a segmentation fault. Your task is to use GDB to find the line number of the wrong statement. Please make use of the GDB commands provided above in order to find and fix the bug.

Task 5: Booting 101

The goal of this part of the lab is to learn about how a computer boots, and work with a dummy operating system! The question of interest here is, when a machine is powered on, how does it load the operating system and its components? where is the kernel stored? which files to read and execute? etc.

The answer to this lies with the idea of loading a portion of data from disk in memory, and executing the corresponding contents. The road to world peace via operating systems starts here. If we can find this special block of data on disk and make sure that the contents of the disk contain the codes for world peace, we are all set. In other words, this special block is the entry point to seize control of the hardware and for the operating system to perform its magic.

When a computer starts, a special program called the Basic Input/Output System (BIOS) is loaded from a chip into the main memory. The BIOS detects connected hardware devices, resets them, tests them etc. and also looks for the special sector (the boot sector) on available disks to load the operating system.

The BIOS reads the first sector of each disk (one by one) and determines whether it is a boot disk (a disk with an operating system). A boot disk is detected via a magic number **0xaa55**, stored as the last two bytes of the boot sector of a disk.

- I. The `boot_sector1.asm` file, in the `task5/` directory, shows a sample assembly code that is supposed to do something. The idea is that this program produces machine instructions that would be copied on the boot sector when the computer is powered-on.

Convert assembly (mnemonics) code to binary using the following,

```
$ nasm boot_sector1.asm -f bin -o boot_sector1.bin
```

If you want to see what is exactly inside the binary file, the following command will help you.

```
$ od -t x1 -A n boot_sector1.bin
```

The above binary can be used to set up (copy to) the first 512 bytes (the boot sector) of a disk. Instead of writing this boot sector to a physical hard disk, we can use an emulator. QEMU is a system emulator that provides a simple and nice method to load and execute the boot sector directly from the bin file.

```
$ qemu-system-i386 boot_sector1.bin
```

The above command emulates a system using the file provided as the attached disk (which in our case has the first 512 bytes of interest).

Compare the outputs of the booting process using the two programs, `boot_sector1.asm` and `boot_sector2.asm`, and justify your results. Submissions should contain binary files and screenshots of QEMU along with an explanation.

- II. Let's do something slightly more interesting. On boot, our custom OS should print out a message.

Write a program, `hello.asm`, that prints custom text (your name?) on the screen during boot-up, for example — “`BuzzLightyear`”.

To print a character on the screen, use the following code with appropriate repetitions and changes.

```
mov ah, 0x0e      ; set tele-type mode (output to screen)
mov al, 'B'        ; one ascii character hex code in register AL
int 0x10           ; send content of register to screen via an interrupt
```

Setup `hello.bin` as the input file for QEMU to use for booting and test output (capture screenshot and save in a file named `hello.png`.)

Submission Guidelines

- All submissions via Moodle.
- Place all the files in your submission directory, with the directory name being your roll number.
- Tar and gzip the directory using the command

```
$ tar -zcvf <rollno_lab1>.tar.gz <rollno_lab1>
```

For example; if your roll number is 123456789, then the directory name will be **123456789**, and submission will be **123456789.tar.gz** and if your roll number is 12D345678, then the directory name will be **12D345678**, and submission will be **12D345678.tar.gz**

- The tar should contain the following files in the following directory structure:

```
<rollnumber_lab1>/
|__task2/
|    |__hellouniverse/
|    |    |__hu.c
|    |    |__Makefile
|    |__helloworld
|    |    |__hw.c
|    |    |__helloworld.h
|    |    |__helloworld.c
|    |    |__Makefile
|    |__Makefile
|__task3/
|    |__exercises_1_to_6.pdf
|__task4/
|    |__debugging_notes.pdf
|__task5/
|    |__boot_sector1.bin
|    |__boot_sector1.png
|    |__boot_sector2.bin
|    |__boot_sector2.png
|    |__hello.asm
|    |__hello.bin
|    |__hello.png
```

- **Deadline: 7th August 2026, 5 pm. (via Moodle)**
-

[OPTIONAL]

Object Files

An object file is a file binary information (object code) a sequence of hardware instructions representing a program. Object files store information about data and code and also information about sections (text, data, etc.), and information used to relocate code and data of binary. An object file is generated by a compiler or an assembler and represents an executable file or a shared library.

ELF (Executable and Linkable Format) is an universally used file format for object files on Unix-like machine, More about ELF here: <https://wiki.osdev.org/ELF>

We will use `objdump` to analyze and understand object file formats and information.

- **objdump**

`objdump` command in Linux is used to provide thorough information for object/exectuable files.

In the folder `object`, find a binary (executable) program named `function.out`.

the file is made by a

company for their aptitude test. It takes

The program takes an input as variable x and computes and output $y = f(x, a, b, c, d)$.

a, b, c, d are secret parameters and are stored as global variables in the program.

The task is to find values of the parameters a, b, c, d .

Use the `objdump` tool to analyze the secret binary file and see if you can figure out the values easily without using your “Math” skills.

Sample `objdump` commands:

- Display all sections' information of the object file.
`objdump -h <object-file>`
- Display the symbol table entries of the object file.
`objdump -t <object-file>`
- Display all headers information.
`objdump -x <object-file>`
- Display all the assembler contents of the executable section.
`objdump -d <object-file>`
- Display the contents of all sections.
`objdump -s <object-file>`
- Display the content of a specific section in hex representation.
`objdump -s -j <section-name> <object-file>`